Simulador de substituição de páginas



Sistemas Operativos 2026/27 Trabalho Prático 2 7 de novembro de 2026

Autor: Francisco Barreiros Vitorino
Número de aluno: a22503547

Repositório GitHub:
github.com/franciscobarreirosvitorino-ai/sistemasoperativos

1 Objetivo e enquadramento

O trabalho compara algoritmos de substituição de páginas num simulador de memória virtual. Uma página é um bloco da memória virtual de um processo; uma frame é um espaço de igual dimensão na RAM. Se uma referência pedir uma página ausente da RAM, ocorre uma falta de página. Quando é necessário libertar espaço, o algoritmo escolhe a página residente a expulsar.

As métricas analisadas são o número de faltas e as operações de swap: leituras do disco para a RAM (swap-in) e escritas da RAM para o disco (swap-out). Uma página alterada, assinalada pelo bit dirty, exige escrita quando é expulsa. Uma página limpa com cópia em swap pode ser descartada sem nova escrita.

2 Algoritmos implementados

Algoritmo	Critério de seleção da vítima
FIFO (fornecido)	Expulsa a página carregada há mais tempo. Um acesso não altera a ordem de entrada.
RANDOM	Escolhe uma entrada residente aleatoriamente; usa rejeição para evitar o enviesamento do resto de rand().
LRU	Escolhe o menor last_accessed. Um contador lógico global distingue referências feitas no mesmo milissegundo.
NRU	Usa um bit de referência, limpo antes dos acessos 5, 9, 13, etc. Procura R=0 desde o início da lista; se todos forem 1, escolhe a primeira entrada.
CLOCK	Percorre circularmente a lista. Limpa R=1 e dá segunda oportunidade; expulsa a primeira entrada com R=0. Conserva o ponteiro para a procura seguinte.

A seleção é feita com --vm FIFO|RANDOM|LRU|NRU|CLOCK. O estado de decisão reside em eviction_order: frame, instante lógico e use_bit. NRU usa a variante de um bit, com períodos de quatro referências globais; não usa classes (R,D). O enunciado não fixa o período ou o desempate.

Fontes: enunciado SO2026-27 T2 Enunciado.pdf; código base fornecido; cenários, testes e registos de execução incluídos na entrega.

Cenários e metodologia

3 Condições de execução

Foram realizadas dez simulações completas: dois cenários, cada um com os cinco algoritmos. Em todas, utilizou-se escalonamento FIFO e uma aplicação, com 5 páginas virtuais, 4 frames físicas e threshold 1. As medições guardadas foram obtidas em Ubuntu 24.04 no WSL, com GCC. A aplicação fornecida não foi alterada.

4 Escolha e explicação dos cenários

Localidade - cenarios/localidade.csv

```
10,0,0,[1,2,3,1,4,1,2,5,1,2,3,4,5]
```

O cenário combina reutilização de páginas, sobretudo 1 e 2, com a entrada de cinco páginas distintas para quatro frames. Foi escolhido para confrontar ordem de entrada, recência e referências recentes. No trecho 1,2,3,1, LRU atualiza a recência da página 1, enquanto FIFO conserva a sua posição. As páginas 4 e 5 criam pressão sobre a memória; o final volta a pedir todas as páginas e pode exigir retornos do swap.

Escritas - cenarios/escritas.csv

```
10,0,0,[-1,2,3,-4,1,2,-1,5,2,3,1,4,5]
```

Foi escolhido para complementar as leituras com escritas explícitas e repetidas. Os valores -1 e -4 escrevem nas páginas 1 e 4. Uma página que regressa do swap fica limpa; uma nova escrita volta a marcá-la como suja. Assim, o segundo -1 permite observar a necessidade de voltar a guardar a página 1 se for novamente expulsa. A alternância cria oportunidades de expulsão e retorno.

Em ambos os CSV, 10,0,0 significa 10 ms de CPU, 0 ms de bloqueio e nice 0. Cada lista contém 13 referências. Valores positivos representam leituras e valores negativos representam escritas na página indicada pelo valor absoluto.

5 Relação com o enunciado

Tarefa	Cobertura
2: conceber inputs	Dois cenários novos, com uma aplicação. O enunciado permite uma ou mais aplicações e não impõe sequências concretas.
3 e 4: executar e demonstrar	Cinco algoritmos com escalonamento FIFO. Registos de expulsões e testes de vítimas exatas complementam as sequências.
7: comparação	Tabelas com faltas, leituras e escritas de swap, baseadas nos registos guardados.
5 e 6: opcionais	RR e simulações com RR e três aplicações não foram implementados.

Resultados e discussão

6 Medições por cenário

Localidade

Algoritmo	Acessos	Faltas	Swap-in	Swap-out
FIFO	13	9	4	4
RANDOM	13	9	4	3
LRU	13	9	4	5
NRU	13	10	5	5
CLOCK	13	9	4	4

FIFO, LRU e CLOCK tiveram 9 faltas, mas LRU fez 5 escritas em swap, contra 4 de FIFO e CLOCK. A igualdade nas faltas não implica igualdade nas operações de disco: as páginas escolhidas e o seu estado dirty também contam. NRU teve 10 faltas e 10 operações de swap no total. RANDOM teve 9 faltas e 7 operações de swap nesta execução.

Escritas

Algoritmo	Acessos	Faltas	Swap-in	Swap-out
FIFO	13	11	6	6
RANDOM	13	8	3	3
LRU	13	11	6	6
NRU	13	11	6	6
CLOCK	13	11	6	6

FIFO, LRU, NRU e CLOCK tiveram 11 faltas, 6 leituras e 6 escritas de swap. RANDOM teve 8 faltas e 6 operações de swap no total. Estes valores de RANDOM correspondem a uma única execução com semente baseada na hora, não a uma média; não permitem concluir que o algoritmo é superior em geral.

7 Interpretação e limites

Nas duas sequências existem cinco páginas distintas. As primeiras cargas contam como faltas, mas não como leituras de swap. Isso explica a diferença de cinco entre o número de faltas e o número de swap-in em cada linha. As páginas novas nascem dirty no código base, mesmo numa leitura inicial, pelo que o cenário de localidade também produz escritas.

Foi mantida a política original de verificar o threshold antes de cada referência, inclusive antes de um hit. Com quatro frames e threshold 1, uma referência pode provocar uma expulsão preventiva. Os valores não devem ser comparados diretamente com exercícios que só substituem páginas em faltas. No NRU, essa expulsão ocorre antes da eventual limpeza de bits dentro de page_request.

Os cenários não foram concebidos para garantir a vitória de LRU e não isolam todas as decisões de NRU e CLOCK. As contagens iguais são resultados válidos; os testes de vítimas exatas são necessários para complementar a evidência. Mais sequências, diferentes números de frames e repetições de RANDOM permitiriam uma comparação mais abrangente.

Validação e reprodução

8 Alterações ao código base

Em virtmem.c foram acrescentadas as quatro funções de substituição, a lista completa de nomes e os casos de despacho. Em virtmem.h foram declaradas as funções. O ponteiro CLOCK é ajustado quando uma entrada é removida. As expulsões imprimem algoritmo, frame, processo, página e dirty.

Foram também corrigidos problemas associados: a página virtual vpn passa a usar ptes[vpn-1], pois o CSV começa em 1; os descritores de frames expulsas perdem a ligação à página e ao processo antigos; uma falta com memória cheia liberta uma frame, permitindo threshold 0. Em ossim.c, thresholds superiores ao número de frames são rejeitados. Estas correções não eram tarefas autónomas explicitamente pedidas no enunciado.

9 Testes realizados

Os testes em test_vm.c verificam seleção e rejeição de nomes, listas vazias, uma única frame, vítimas FIFO/LRU/NRU/CLOCK, segunda oportunidade, continuidade do ponteiro CLOCK, propriedades de RANDOM e limpeza periódica NRU. Verificam ainda limites da tabela, ciclos RAM/swap e ausência de dupla libertação após expulsão.

Os testes passaram com GCC, AddressSanitizer e UndefinedBehaviorSanitizer. Foram verificados argumentos inválidos e executadas as dez simulações completas. As evidências estão em resultados/validacao.txt, resultados/comparacao.csv e nos registos de cada cenário. A configuração CMake foi preparada, mas a compilação efetivamente validada foi com GCC, por não existir CMake no ambiente de teste.

10 Compilar e repetir

Na pasta do projeto, num ambiente Linux/POSIX:

```
mkdir -p build
gcc -Wall -Wextra -Wno-unused-parameter ossim.c queue.c
  scheduler.c virtmem.c -o build/ossim
gcc -Wall -Wextra -Wno-unused-parameter application.c
  burst_queue.c -o build/application
```

Nos comandos anteriores, juntar as duas linhas de cada comando gcc numa única linha. Para executar automaticamente as dez combinações: **python3 comparar.py**. Não iniciar outros simuladores ao mesmo tempo: usam o mesmo socket Unix.

Para uma execução manual, iniciar o simulador num terminal e a aplicação noutro:

```
./build/ossim --sched FIFO --vm LRU --vpages 5
  --pframes 4 --threshold 1
./build/application cenarios/localidade.csv
```

Juntar as duas primeiras linhas num só comando. Após as estatísticas, terminar o simulador com Ctrl+C e reiniciá-lo para a combinação seguinte. Com CMake disponível: cmake -S . -B build; cmake --build build; ctest --test-dir build --output-on-failure.

11 Conclusão

Foram implementados RANDOM, LRU, NRU e CLOCK e comparados com FIFO. Os resultados mostram que a escolha da vítima afeta tanto as faltas como as operações de swap e depende da sequência e da política de reserva de frames. A validação combina simulações completas com testes de decisões específicas. As tarefas opcionais com RR ficaram fora do âmbito.